

Predicting Coffee Yield from Agroecological and Landscape Features

By Adia Redd and Tiffany Tang

Abstract

This study aimed to identify which agroecological or landscape feature best predicts mean coffee yield in Brazil, with the initial hypothesis that forest cover would be the most influential predictor. Using 51 environmental and landscape variables, we conducted exploratory analyses including histograms, correlation and principal component analysis to characterize data structure and multicollinearity. Preprocessing steps included label encoding, skewness-based log transformation, feature scaling and dimensionality reduction, which resulted in three dataset variants: a scaled/log-transformed, the same data set that was then PCA- transformed, and a reduced feature data set that was scaled, log transformed and PCA- transformed.

We evaluated Linear Regression, Random Forest Regression and Support vector Regression using GridSearch CV and assessed performance with R^2 , error metrics, and residual plots. Predictive performance was modest across all approaches, with the best models achieving test R^2 values of ~ 0.25 – 0.30 . PCA and feature reduction generally reduced accuracy, suggesting that important signals were lost. SHAP analyses revealed that regional identifiers and temperature-related variables were the most influential predictors across models, while forest cover did not appear prominently in any model or principal component. Feature importance patterns varied across algorithms, limiting interpretability and model reliability.

Overall, results indicate that the current feature set and models are insufficient for accurately predicting coffee yield or identifying a dominant ecological driver. Future directions include exploring neural networks, gradient boosting, spatial modeling, and alternative dimensionality reduction methods to better capture complex environmental relationships.

Introduction

Coffee production is a major global agricultural sector and Brazil is its largest producer, supplying roughly one-third of the world's coffee. However, coffee yields can vary substantially across regions due to differences in climate, landscape structure, and ecological conditions. Prior ecological research has suggested that forest cover and landscape heterogeneity can enhance coffee productivity by improving pollination, stabilizing microclimates, and supporting natural pest control (De Marco & Coelho, 2004; Jha et al., 2014). These findings motivate a more quantitative assessment of how agroecological and landscape variables relate to yield across broader spatial scales.

The dataset used in this study (Silva et al., 2021; Zenodo record 5574892) combines municipal-level coffee yield with 51 environmental, climatic, and landscape features, including forest extent, temperature metrics, moisture indices, crop cover, topography, and spatial identifiers. Using this dataset, our goal is to answer the following research question: Which agroecological or landscape features best predict mean coffee yield across Brazilian farms? Based on previous ecological studies, our hypothesis is that forest cover will be the strongest predictor of yield.

Identifying the key drivers of coffee productivity is an important problem for both ecological and agricultural planning. Accurate prediction models can help farmers and policymakers anticipate yield constraints, evaluate the benefits of forested landscapes, and improve management under climate variability. To address this, we apply multiple machine learning models, Linear Regression, Random Forest Regression, and Support Vector Regression, along with extensive preprocessing, PCA, and SHAP-based interpretability to determine which variables consistently contribute to predictive performance.

Methods

Dataset and Research Design

We used the publicly available agroecological dataset from Silva et al. (2021) (Zenodo record 5574892), which contains mean coffee yield and 51 landscape, climatic, and ecological variables from Brazilian municipalities. Our objective was to build predictive models of mean yield and determine which features contribute most strongly to prediction accuracy, with a specific hypothesis that forest cover would be a major predictor.

Data Preprocessing

Three categorical variables were label-encoded. Numerical features were screened for skewness using Pandas' `.skew()`; variables with absolute skewness > 1.0 were considered for transformation, excluding binary, near-constant, and negative-valued variables. A natural $\log(+1)$ transformation was applied only when it reduced skewness. To avoid data leakage, all datasets were split into training and testing sets (80/20) before scaling or PCA. Three preprocessing pipelines were evaluated: Log-Transformed + Scaled Dataset; Log-Transformed + Scaled + PCA Dataset; Correlation/Variance-Reduced + Log-Transformed + PCA Dataset; For Pipeline 3, features with correlation > 0.9 or variance < 0.01 were removed prior to PCA.

Model Training

We trained three families of regression models on each dataset variant: Ordinary Linear Regression; regularized models (Ridge, Lasso, ElasticNet); Random Forest Regression (RFR); and Support Vector Regression (SVR). Hyperparameters for all non-ordinary models were optimized using GridSearchCV with five-fold cross-validation. For RFR, the tree-splitting

criterion and the maximum number of features considered at each split were tuned, with `random_state=8` and out-of-bag error estimation enabled. For the SVR model, we performed a grid search over the `C`, `gamma`, and `epsilon` hyperparameters to identify the best-performing configuration of the RBF kernel. The RBF kernel was selected because it can model non linear relationships, which are expected in ecological yield data.

Model Evaluation

Performance was assessed using R^2 , mean absolute error (MAE), and root mean squared error (RMSE), which are standard metrics for regression prediction. Residual plots were generated for all models to evaluate residual behavior and potential patterns. Quantile–quantile (QQ) plots were generated for the linear regression models to assess normality assumptions. Feature importance and interpretability were assessed using SHAP values for each best-performing model on each dataset variant.

Reproducibility

All analyses were conducted in Python. Full preprocessing, modeling and visualization code is included below in Appendix A to ensure complete reproducibility.

Results

Exploratory Data Analysis

Initial visualization of all 51 agroecological and landscape features (Fig. 1) revealed substantial non-normality across many predictors. Several variables exhibited heavy right skew, prompting the use of log-transformations to reduce extreme values and improve model stability. A $\log(x + 1)$ transformation was applied to the subset of highly skewed continuous features (Fig. 1). Although this transformation shifted some variables toward more symmetric distributions, most remained moderately or strongly skewed (Fig. 3a–b). The log transformation offered only partial improvement and did not meaningfully change the overall distributional structure of the dataset.

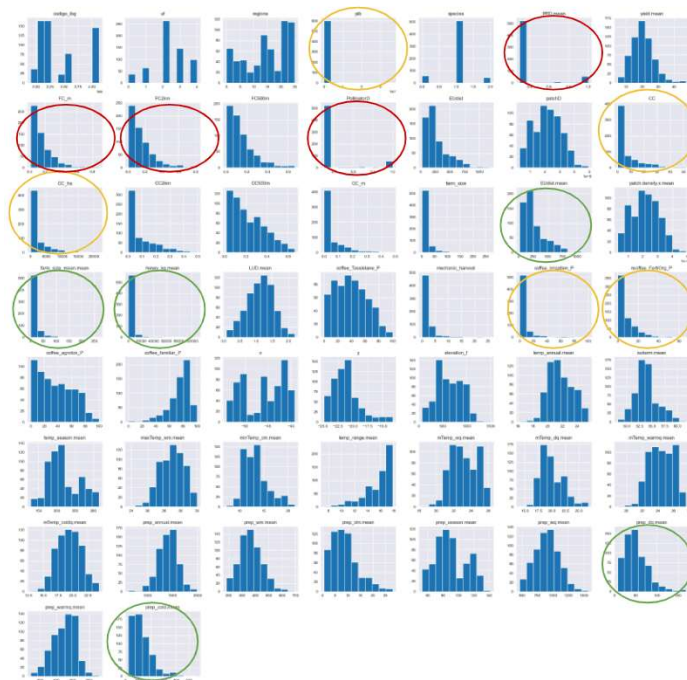


Fig 3a. Some Features Before Log Transformation

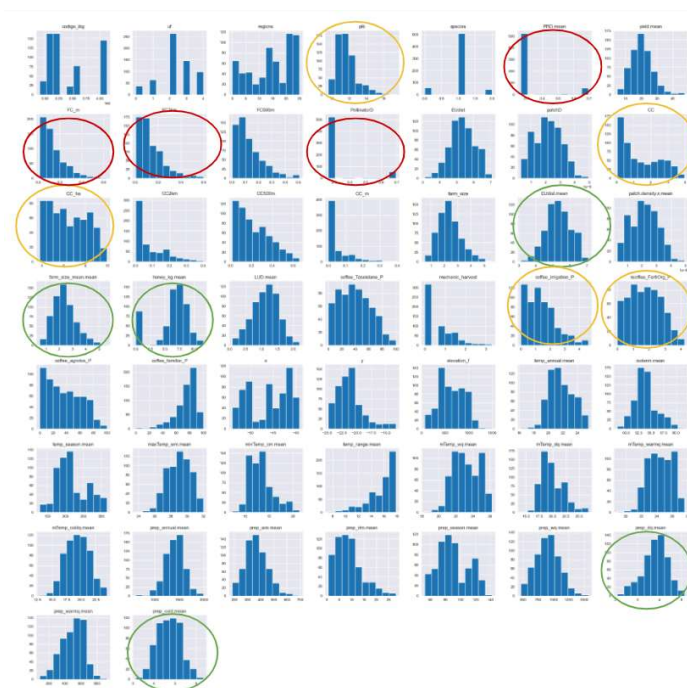


Fig 3b. Some Features After Log Transformation

A correlation heatmap (Fig. 2) revealed three major multicollinearity clusters: (1) vegetation and field-cover metrics, (2) crop-cover indicators, and (3) temperature-related variables. These clusters indicate substantial redundancy among predictors that likely capture similar underlying ecological processes. This motivated the creation of a reduced-feature dataset where highly correlated variables and features with 0 variance were removed upfront before being PCA transformed.

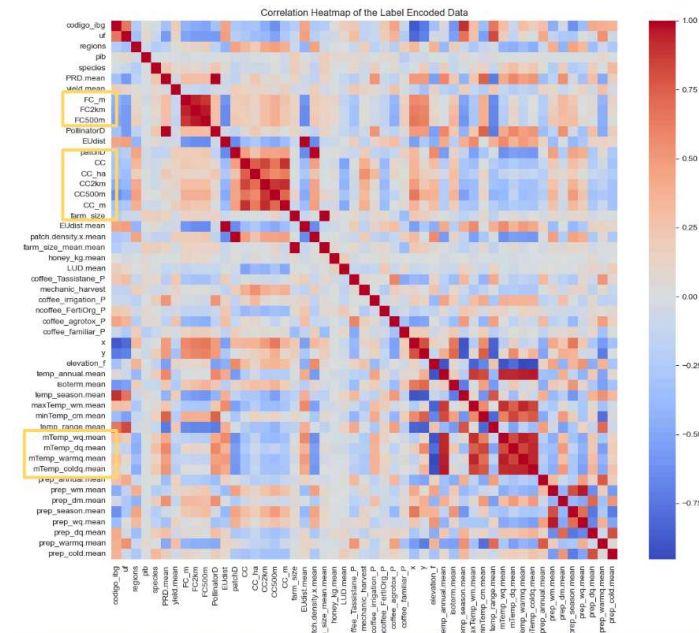


Fig 4. Correlation Heatmap

Principal Component Analysis (PCA) was performed on the scaled dataset to evaluate underlying structure. In Figure 5 we see that The first 20 principal components explained approximately 95% of total variance, with an elbow at three components (55% cumulative variance). However, initial modeling showed that three components were insufficient; therefore, the first 7 PCs (75% cumulative variance) were retained for PCA-based models. Scatterplots of samples projected onto PC1–PC2 (Fig. 6) and PC1–PC3 (Fig. 7) showed no visible clustering by average yield, suggesting limited separability of yield levels in a linear subspace.

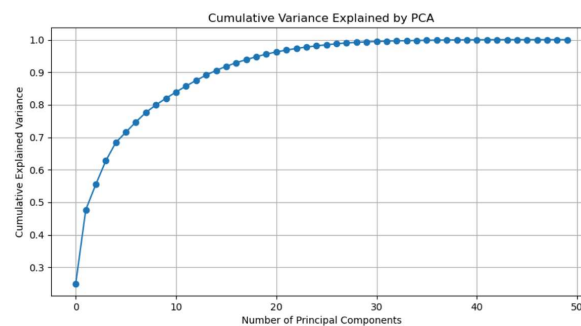


Fig 5. Cumulative Variance

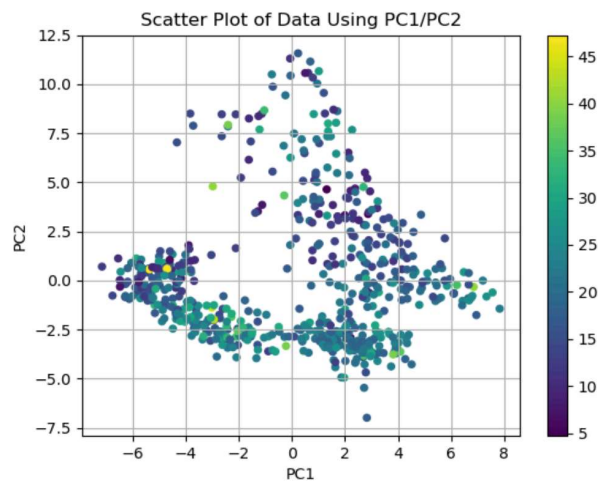


Fig 6. PC1 vs PC2 Scatter

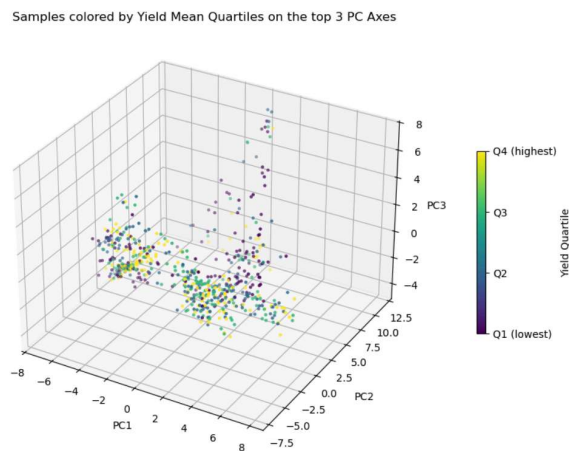


Fig 7. 3D PCA Scatter

To evaluate the effect of preprocessing and dimensionality reduction on predictive performance, three dataset variants were constructed:

First Dataset (No PCA)

- This version retained all original 51 predictors after scaling and selective log transformation. It served as the baseline for comparison and allowed for direct interpretation of feature coefficients and importance scores.

Second Dataset (7 PCs)

- Principal Component Analysis was applied to the scaled data (after selective log transformation), and the first seven principal components, which explain approximately 75% of the total variance, were selected (Figs. 3–4). This representation aimed to reduce collinearity and compress the environmental signal into a smaller set of orthogonal predictors.

Third Dataset

- Highly correlated (>0.9) and near-zero-variance predictors were removed prior to scaling and PCA. This version evaluates whether feature pruning followed by dimensionality reduction yields more stable or interpretable models.

All models were trained on an 80/20 train–test split using standardized inputs, including the random forest regressor (RFR), even though scaling is not required for tree-based models. This ensured a fair, consistent comparison across approaches while preserving proper separation of training, validation, and test data.

Linear Regression

Non-PCA Model

Ordinary Linear Regression achieved a test R^2 of approximately 0.11, indicating that the raw feature space provides very limited linear signal for predicting coffee yield. Introducing regularization improved cross-validated R^2 values: Ridge, Lasso, and ElasticNet models reached ~ 0.20 – 0.21 during training, but their performance declined on the held-out test set (test $R^2 \approx 0.12$ – 0.15). This pattern suggests mild overfitting, where regularization helps stabilize coefficients but cannot uncover strong linear structure in the data (Fig. 8).

```
Linear Regression R²: 0.11046279065026987
MSE: 40.33039940799774
Best Ridge R²: 0.2131117522415525
Best Ridge Params: {'alpha': 10}
Test Ridge R²: 0.12676760532317133

Best Lasso R²: 0.20321081766870916
Best Lasso Params: {'alpha': 0.01}
Test Lasso R²: 0.11977609391137412

Best ElasticNet R²: 0.21199891277155264
Best ElasticNet Params: {'alpha': 0.1, 'l1_ratio': 0.1}
Test ElasticNet R²: 0.14038942171086177
```

Fig 8. Linear Regression First Dataset Output

PCA Models

Applying PCA did not improve linear model performance. When trained on the first seven principal components, all linear models, including regularized variants, performed slightly worse than their non-PCA counterparts. Test R^2 values generally fell below 0.10, indicating that dimensionality reduction removed variance that was weakly but genuinely related to yield, without revealing any clearer linear structure.

These results suggest that although PCA effectively compresses the feature space, the principal components do not align with directions that support linear prediction of coffee yield. In other words, the dominant sources of variance in the landscape and ecological data are not the same sources of variance that drive yield differences.

```

Linear Regression R²: 0.0853147297506559
MSE: 41.470578064671926
Best Ridge R²: 0.10388214839689862
Best Ridge Params: {'alpha': 100}
Test Ridge R²: 0.08515084431070541

Best Lasso R²: 0.10260621746496576
Best Lasso Params: {'alpha': 0.1}
Test Lasso R²: 0.09032734414861887

Best ElasticNet R²: 0.10296714227636464
Best ElasticNet Params: {'alpha': 0.1, 'l1_ratio': 0.1}
Test ElasticNet R²: 0.08580458632558774

```

Fig 9. Linear Regression Second Dataset Output

When highly correlated and near-zero variance features were removed prior to PCA (third dataset), linear model performance deteriorated substantially. Across all regularized and unregularized variants, test R^2 values were near zero or negative, indicating that the models performed worse than a baseline predictor that simply outputs the mean yield. This pattern suggests that the feature-reduction step discarded variables containing meaningful but diffuse signal, leaving the resulting principal components unable to represent relationships relevant for linear prediction. As a result, the models exhibited severe underfitting, reinforcing that linear methods are highly sensitive to information loss introduced during aggressive preprocessing.

```

Linear Regression R²: 0.007988231733874418
MSE: 44.9764556345583
Best Ridge R²: -0.0294557642678547
Best Ridge Params: {'alpha': 100}
Test Ridge R²: 0.008513595729569334

Best Lasso R²: -0.018238564024719173
Best Lasso Params: {'alpha': 10}
Test Lasso R²: -0.002071019158188747

Best ElasticNet R²: -0.018238564024719173
Best ElasticNet Params: {'alpha': 10, 'l1_ratio': 0.3}
Test ElasticNet R²: -0.002071019158188747

```

Fig 10. Linear Regression Third Dataset Output

SHAP Feature Importance

SHAP (SHapley Additive exPlanations) analysis of the linear regression model trained on the full, non-PCA dataset showed that only a small number of variables had moderate influence on predicted yield (Fig. 11). The strongest contributor was `codigo_ibg`, a geographic identifier, indicating substantial spatial structure not fully captured by the agroecological features. Several temperature-related variables (e.g., `isoterm.mean`, `temp_season.mean`, `temp_range.mean`) also contributed, with both positive and negative SHAP values,

suggesting complex or nonlinear relationships. Most features had SHAP values close to zero, consistent with the model's low predictive performance ($R^2 \approx 0.11$ – 0.20). Forest cover variables did not appear among the top contributors, contrary to the original hypothesis.

For the PCA-transformed linear model, feature importance shifted from individual environmental variables to principal components (Fig. 12). PC2 emerged as the most influential component, while PC3, PC4, and PC6 showed moderate contributions. In contrast, PC1, despite explaining the largest share of variance, had little influence on yield predictions. This indicates that components capturing the most overall variance were not necessarily those most relevant for prediction. SHAP values remained small in magnitude overall, consistent with the model's limited performance after PCA (test $R^2 \approx 0.08$ – 0.10).

When PCA was applied after removing highly correlated and near-zero variance features, SHAP patterns changed markedly (Fig. 13). PC1 became relatively more important, while PC2 and PC3 remained modest contributors. However, SHAP magnitudes were smaller overall, aligning with the model's severe underfitting and negative test R^2 values. This suggests that removing correlated features eliminated informative signal, resulting in principal components that were less aligned with yield-related ecological variation.

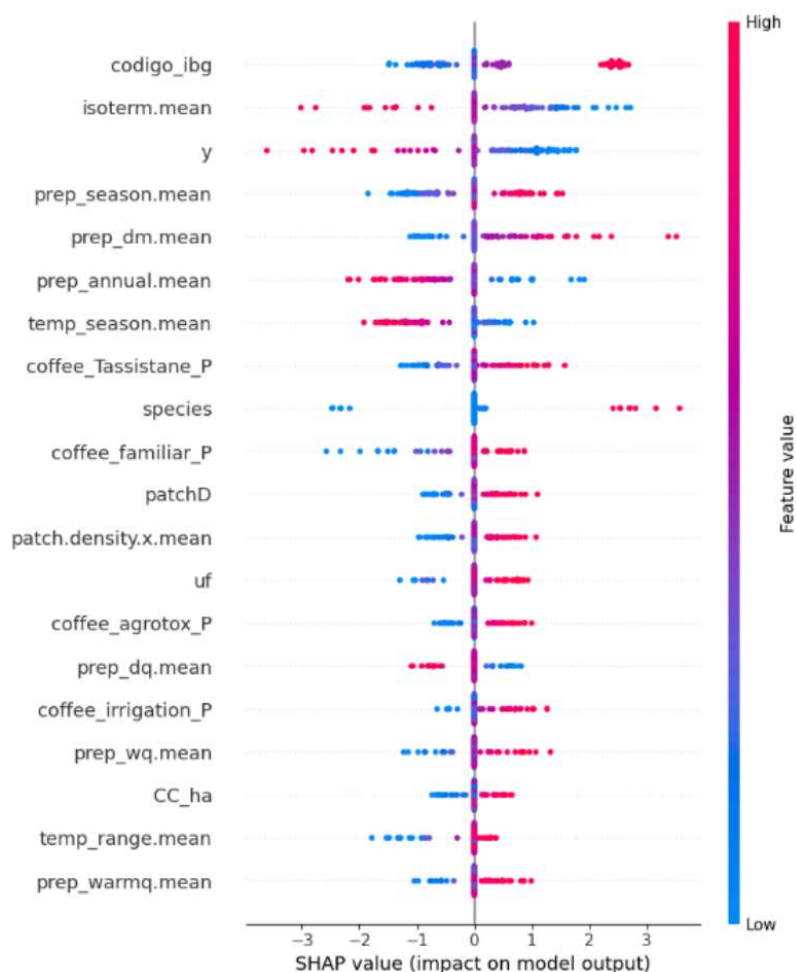


Fig 11. Linear Regression SHAP Summary Plot First Dataset

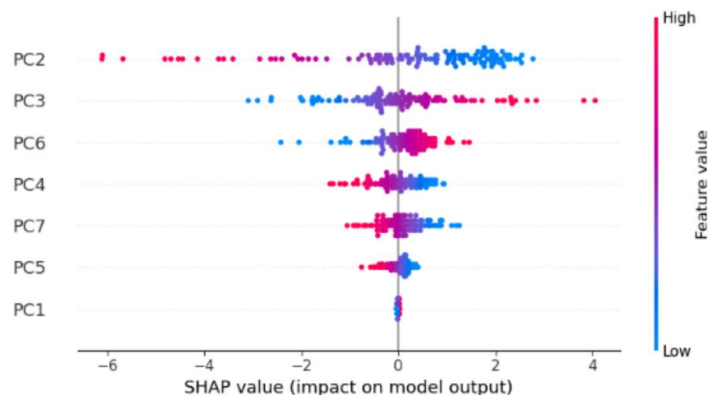


Fig 12. Linear Regression SHAP Summary Plot Second Dataset

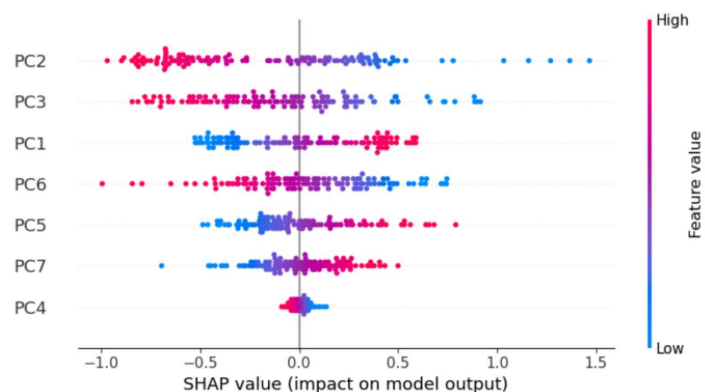


Fig 13. Linear Regression SHAP Summary Plot Third Dataset

Residuals and QQ plot Diagnostics

Residual diagnostics were used to evaluate linear regression assumptions across the three preprocessing approaches: (1) log-transformed and scaled predictors, (2) PCA-transformed predictors, and (3) reduced-feature PCA predictors. The residual-predicted plots (Fig. 14a–c) and QQ plots (Fig. 15a–c) consistently reveal substantial deviations from the assumptions required for reliable linear modeling.

The three residual plots show a largely patternless cloud, indicating no strong violations of linear regression assumptions (e.g., heteroscedasticity or nonlinearity). However, this randomness also reflects the fact that the models are underfitting: the predicted values vary very little while the residuals remain large. The model is not capturing meaningful structure in the data, which produces ‘clean’ residual patterns but poor predictive performance. The residuals remain widely dispersed around zero, and no model shows evidence of improved homoscedasticity following transformation or PCA. In the PCA-based models (Figs. 14b and 14c), the predicted-value range collapses compared to the non-PCA model, reflecting the loss of variance introduced by dimensionality reduction; however, this compression does not generate more well-behaved residuals. Instead, the spread remains large relative to the narrow band of predictions, indicating underfitting.

The QQ plots (Figs. 15a–c) show systematic departures from normality in all cases. While the middle quantiles roughly follow the theoretical line, the tails diverge substantially—particularly the upper tail—which indicates non-normal, heavy-tailed residuals. Neither log-scaling nor PCA succeeded in producing residual distributions resembling a Gaussian, further confirming model misspecification. The reduced-feature PCA model exhibits the largest tail deviations, consistent with its poorest predictive performance.

The residual and QQ plot diagnostics reinforce the quantitative model metrics: linear regression assumptions are violated under all preprocessing approaches, with clear evidence of nonlinearity, heteroscedasticity, and heavy-tailed error structure. These findings confirm that linear regression is poorly suited for modeling mean coffee yield in this dataset and that more flexible nonlinear approaches (e.g., SVR, Random Forest) are better aligned with the underlying data structure.

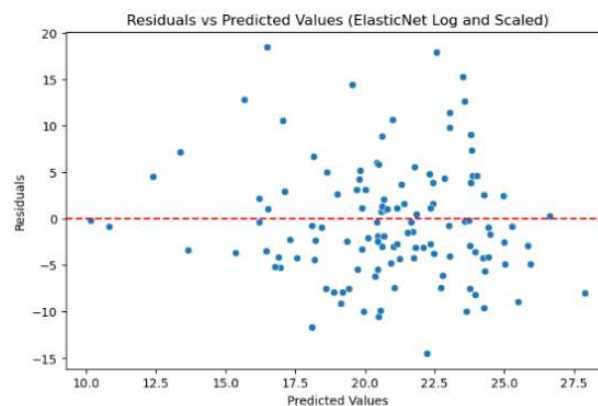


Fig 14a. Linear Regression First Dataset Residuals Plot

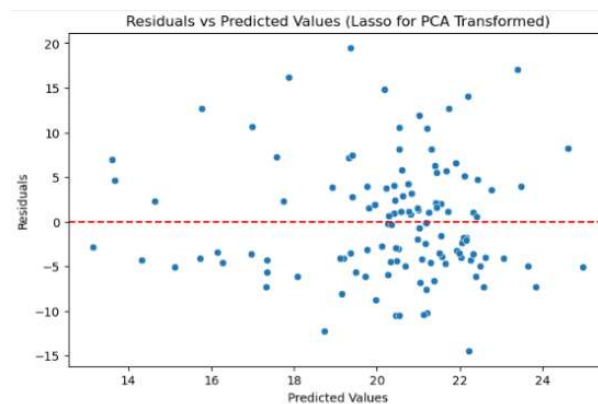


Fig 14b. Linear Regression Second Dataset Residuals Plot

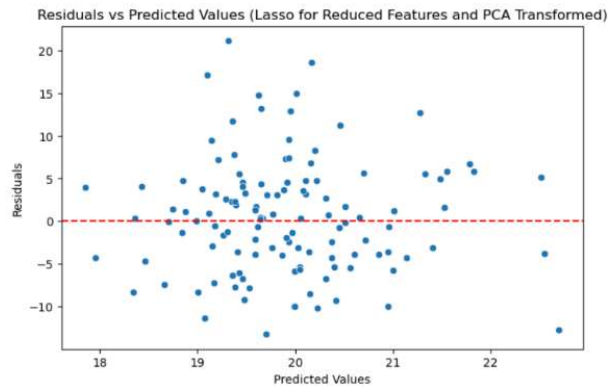


Fig 14c. Linear Regression Third Dataset Residuals Plot

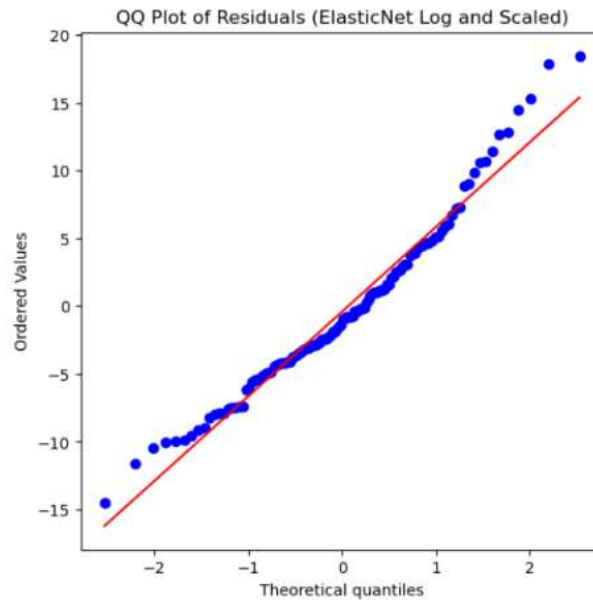


Fig 15a. Linear Regression First Dataset QQ Plot

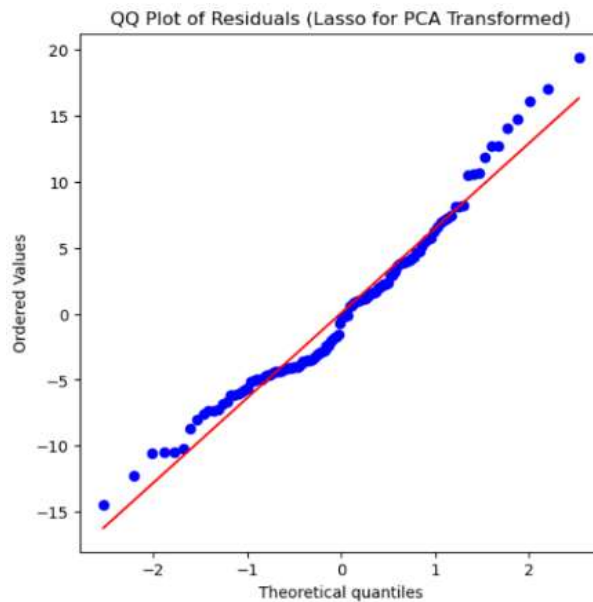


Fig 15b. Linear Regression Second Dataset QQ Plot

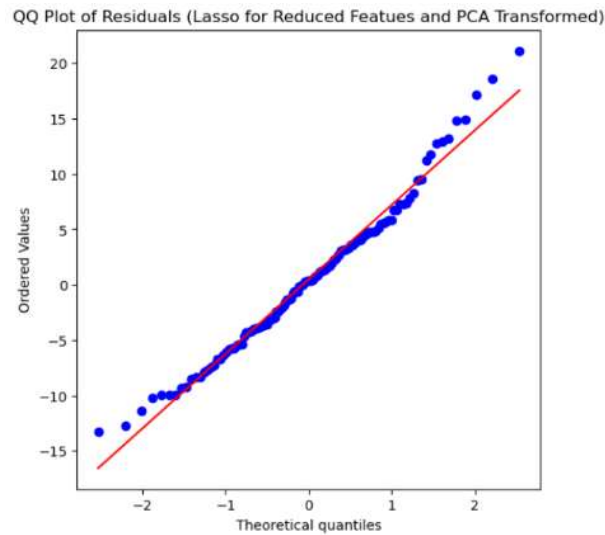


Fig 15c. Linear Regression Third Dataset QQ Plot

Random Forest Regression

Non PCA Model

The best model of all three families is the RFR trained on the data that was simply log-transformed and scaled (dataset variant 1), defined the loss function as squared error and the maximum number of features to be considered when making the split in a given tree equal to the log base 2 of the number of samples (Fig 16a). The R^2 score of this model with the test set is 0.28 (Fig 16b).

```
Best R2: 0.304016481684443
Best Params: {'criterion': 'squared_error', 'max_features': 'log2'}
```

Fig 16a. RFR First Dataset GridSearchCV Output

```
Test R2: 0.2818931059573022
Test MAE: 4.457041666666666
Test RMSE: 5.705960355487387
```

Fig 16b. RFR First Dataset Model Test Output

PCA Models

GridSearch reported the same optimal model when using the remaining two dataset variants for fitting, where the loss function is absolute error and the maximum number of features to be considered when making the split equal to the square root of the number of samples (Fig 17a, 18a). This model returned R^2 scores of 0.28 and -0.05 (Fig 17b, 18b) when tasked with predicting the mean yield using their respective test sets.

```
Best R2: 0.14790220191350834
Best Params: {'criterion': 'absolute_error', 'max_features': 'sqrt'}
```

Fig 17a. RFR Second Dataset GridSearchCV Output

```
Best R2: -0.03531986845134207
Best Params: {'criterion': 'absolute_error', 'max_features': 'sqrt'}
```

Fig 18a. RFR Second Dataset GridSearch CV Output

```
Test R2: 0.2680723708042566  
Test MAE: 4.518573087431693  
Test RMSE: 5.760607328346087
```

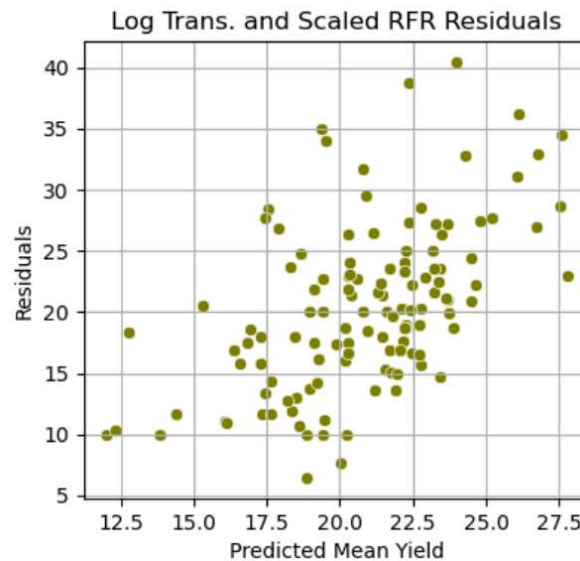
Fig 17b. RFR Third Dataset Model Test Output

```
Test R2: -0.05136853198659708  
Test MAE: 5.417416894353369  
Test RMSE: 6.904173379177494
```

Fig 18b. RFR Third Dataset Model Test Output

Residuals and QQ plots

Despite this poor correlation between the predicted and actual y-values, the residuals plots did not indicate any extremely abnormal behavior for either model. The residuals produced by the top scoring model did, however, display a very mild positive correlation between the actual and predicted mean yield (Fig 19a). This could indicate that the dataset could benefit from a more complex model to explain the relationship between the features and the mean yield since there may be a violation of the assumptions associated with regression. The remaining two models produced residual plots with no discernible pattern (Fig 19b, 19c). These observations indicate that the model is capable of capturing some of the variability in the data, but loses important information that is likely stored in higher dimensions.

**Fig 19a.** Residual Plot of RFR Model Predictions for First Dataset

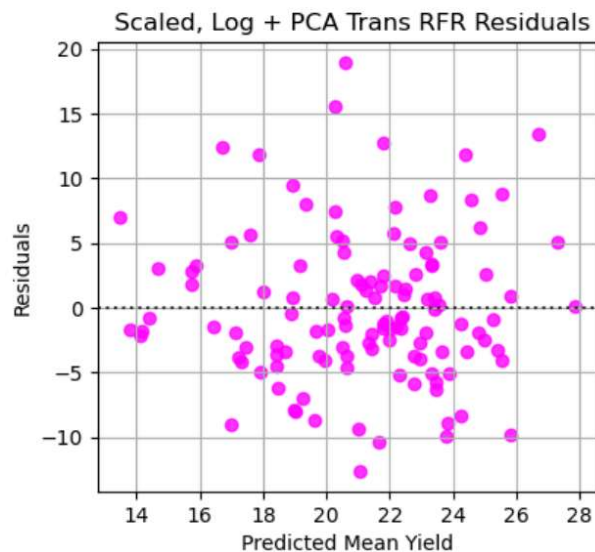


Fig 19b. Residual Plot of RFR Model Predictions for Second Dataset

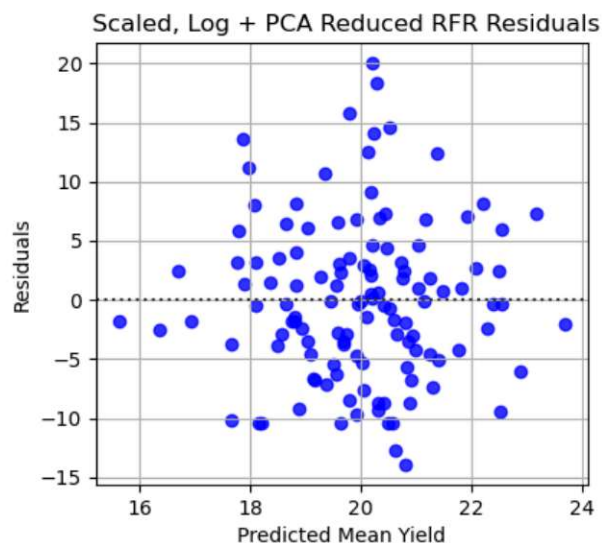


Fig 19c. Residual Plot of RFR Model Predictions for Third Dataset

SHAP Feature Importance

SHAP summaries were generated for both models to investigate which feature was the most important in the model's decision to predict a value for a given sample. The first model's SHAP summary (Fig 20) reported the top five most important features to all be temperature-related, aside from argotox_P, which is the percentage of farms in the municipality that used the ArgoTox plant protection agent.



Fig 20. RFR SHAP Summary for First Dataset

For all features there is no clear separation of high and low values in terms of SHAP value. This indicates that for most samples, the model had a difficult time partitioning a sample into one node or the other when a split in a tree arose because it could not be determined if a high or low value was important or not. When predicting y-values for the second dataset variant's test split, the second model selected PC2 as it's most important feature and PC1 as it's least important (Fig 21). When predicting the y-values for the third dataset variant's test split, PC1 was selected as most important and PC7 as least important. Despite being the same model, the important components changes drastically (Fig 22). Since PC2 was indicated a number of times across the models, it will be discussed separately. PC1 is composed mostly of geographic features like Euclidean distance between forest fragments and coffee fields, longitude/latitude, and municipality identifiers.

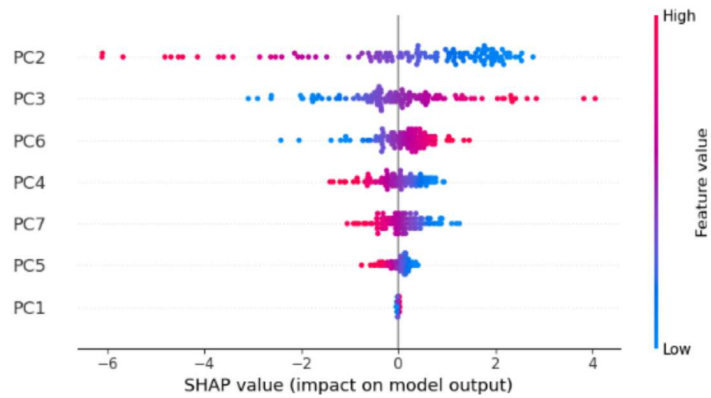


Fig 21. RFR SHAP Summary for Second Dataset

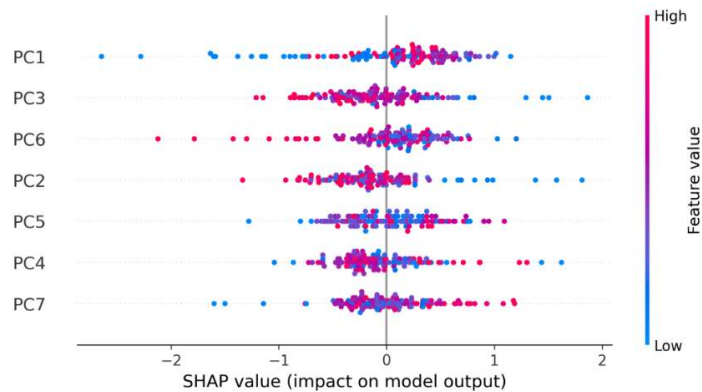


Fig 22. RFR SHAP Summary for Third Dataset

Support Vector Regression

Non-PCA Models

Among the SVR configurations evaluated, the non-PCA model trained on the first dataset achieved the strongest overall performance, with an R^2 of approximately 0.25 on the training set and 0.28 on the test set (Fig. 23a–b). The similarity between training and test performance indicates stable generalization and little evidence of overfitting. Corresponding error metrics (MAE $\approx$ 4.39; RMSE $\approx$ 5.73) suggest moderate predictive accuracy, indicating that SVR was able to capture some nonlinear relationships between agroecological variables and mean coffee yield.

```
Best R2 training: 0.254393067638455
Best Params: {'svr_C': 10, 'svr_epsilon': 1.0, 'svr_gamma': 'scale'}
```

Fig 23a. SVR First Dataset Training Output

```
Test R2: 0.2757089354011931
Test MAE: 4.394429170623653
Test RMSE: 5.730476891678431
```


Fig 23b. SVR First Dataset Testing*PCA Models*

Applying SVR to PCA-transformed inputs resulted in a modest decline in predictive performance, with R^2 values ranging from approximately 0.16 to 0.24 (Fig. 24a–b). Although PCA reduced dimensionality and multicollinearity, restricting the model to seven principal components likely removed predictive structure relevant to yield. Additionally, the nonlinear relationships that SVR is designed to model may not be fully preserved in the orthogonal PCA space, limiting its effectiveness.

```
Best R²: 0.15586788164418672
Best Params: {'C': 10, 'epsilon': 0.1, 'gamma': 0.01}
```

Fig 24a. SVR Second Dataset Training Output

```
Test R²: 0.23863733802790776
Test MAE: 4.552965405910848
Test RMSE: 5.875299216604867
```

Fig 24b. SVR Second Dataset Testing output

SVR performance declined substantially when correlated features were removed prior to PCA, resulting in negative test-set R^2 values (Fig. 25a–b). This indicates that the model performed worse than a baseline mean predictor on unseen data. As observed in other modeling approaches, aggressive feature reduction appears to have eliminated informative signal, suggesting that correlated variables in this dataset still contain meaningful predictive information rather than purely redundant noise.

```
Best R²: -0.0377210885531686
Best Params: {'C': 10, 'epsilon': 0.1, 'gamma': 0.001}
```

Fig 25a. SVR Third Dataset Training Output

```
Test R²: -0.003934873589512167
Test MAE: 5.261275720017252
Test RMSE: 6.7466312266714015
```

Fig 25b. SVR Third Dataset Testing output*Residual Plots*

Residual plots for the SVR models across the three dataset variants are presented in Fig. 26a–c. Across all datasets, residuals are generally centered around zero and display no

strong monotonic trends with respect to predicted mean yield, indicating that the models do not exhibit clear systematic bias. However, residual dispersion remains substantial in all three cases, with a wide range of positive and negative errors persisting across the prediction space. In the first dataset (Fig. 26a), residuals show moderate scatter with several large deviations at both low and high predicted yields. The second dataset (Fig. 26b) exhibits a similarly broad spread of residuals, despite log transformation and feature scaling, suggesting that these preprocessing steps did not substantially reduce prediction error. For the third dataset (Fig. 26c), PCA transformation likewise failed to produce a marked improvement in residual structure, with errors remaining widely distributed and only loosely clustered around zero.

While the residuals appear largely random, their magnitude and dispersion indicate that a considerable portion of yield variability remains unexplained by the SVR models. The absence of clear structure in the residuals therefore reflects limited model fit rather than strong predictive performance, consistent with the relatively low R^2 values observed across datasets.

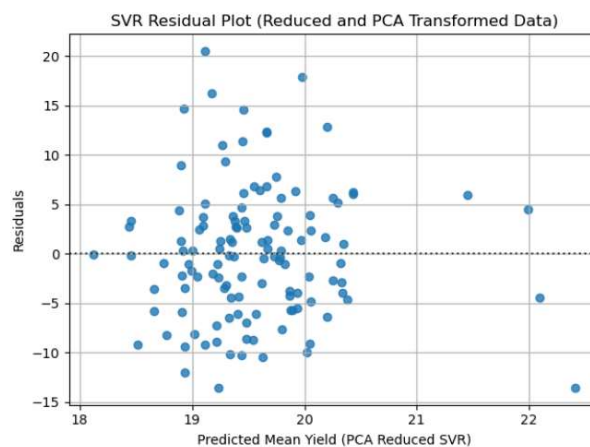


Fig 26a. Residual Plot of SVR Model Predictions on First Dataset

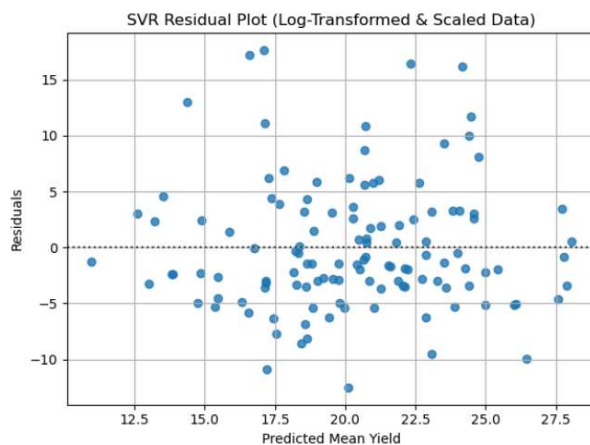


Fig 26b. Residual Plot of SVR Model Predictions on Second Dataset

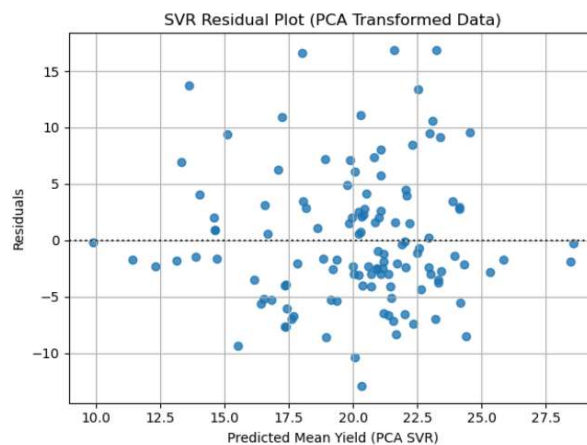


Fig 26c. Residual Plot of SVR Model Predictions on Third Dataset

SHAP Feature Importance

SHAP summary plots for the SVR models trained on the three dataset variants are shown in Fig. 27–29. For the model trained on the full feature set (Fig. 27), the largest SHAP values were associated with the geographic identifier `codigo_ibg` and several climate-related variables, including `isoterm.mean`, seasonal precipitation, and mean temperature. These features showed the widest spread in SHAP values, indicating the strongest influence on predicted yield. Land-use and management variables, such as coffee cover (`CC_ha`), agrotoxic use, and patch density, contributed more modestly, with SHAP values clustered closer to zero.

For the PCA-transformed dataset (Fig. 28), SHAP values were dominated by the first two principal components, with PC2 showing the greatest spread and PC1 contributing to a lesser extent. Higher-order components (PC3–PC7) exhibited substantially smaller SHAP magnitudes, indicating limited influence on model predictions.

In the reduced-feature PCA model (Fig. 29), SHAP values were further compressed toward zero across all components. PC2 and PC1 remained the most influential components, but their SHAP magnitudes were smaller than in the full PCA model, reflecting the reduced contribution of the transformed feature space to SVR predictions.

Across the three datasets, SHAP results show a progression from more distinct feature contributions in the full feature model to increasingly attenuated and compressed importance patterns in the PCA-based models, particularly after feature reduction.

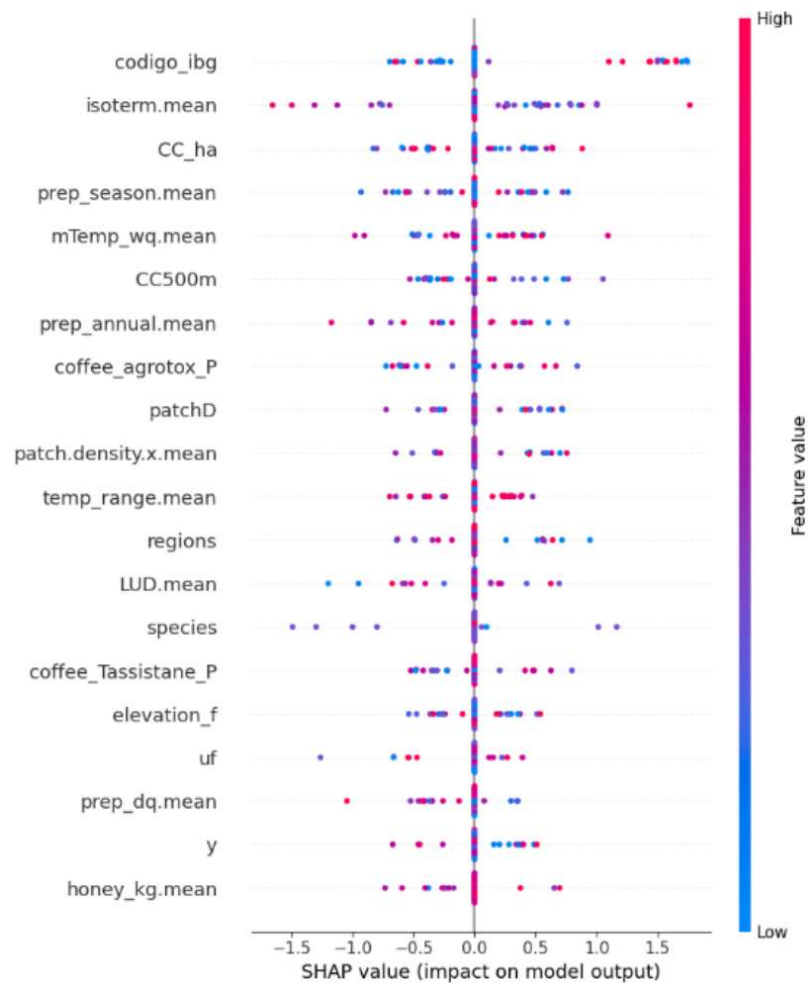


Fig 27. SVR SHAP Summary Plot First Dataset

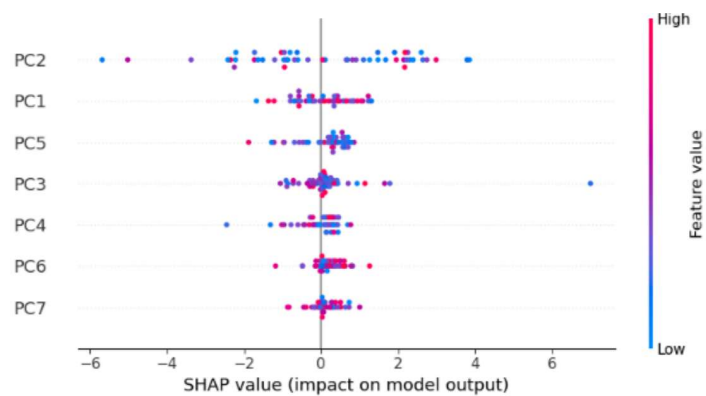


Fig 28. SVR SHAP Summary Plot Second Dataset

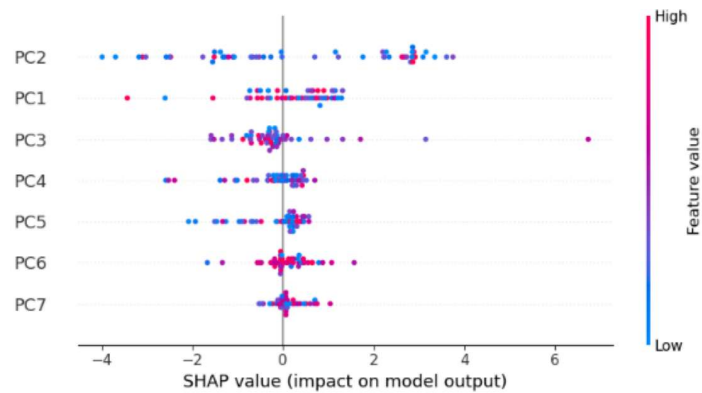


Fig 29. SVR SHAP Summary Plot Third Dataset

Principal Component 2

Since PC2 was repeatedly revealed as an important feature we investigated which features were used to create it. In the full feature PCA, PC2 is dominated by temperature-related variables, along with average pollination activity and latitude, which all load positively (Fig 28a). Elevation, warm-season average temp, and temperature range load negatively. After removing correlated features, the structure of PC2 shifts: elevation becomes the strongest driver, loading positively, and is accompanied by warm season precipitation-related variables, while several cool-season precipitation-related variables load negatively (Fig 28b). Both PC2's contain temperature features, but neither load forest cover into their composition.

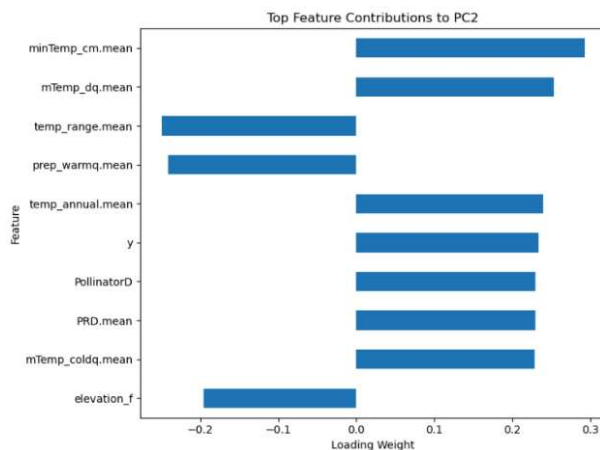


Fig 28a. PC2 loading of Dataset Variant 2

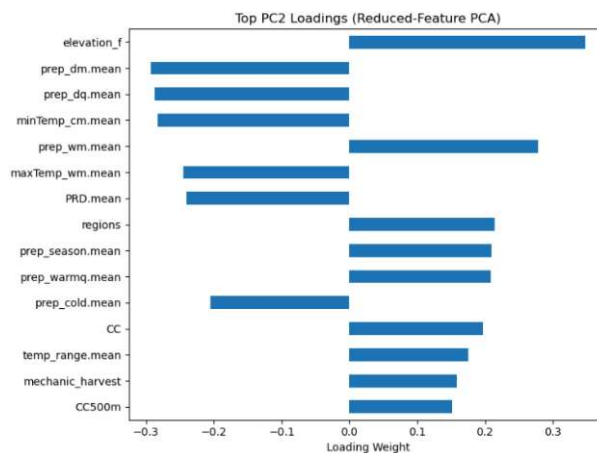


Fig 28b. PC2 loading of Dataset Variant 3

Conclusion and Discussion

The objective of this study was to evaluate whether agroecological and landscape features could reliably predict mean coffee yield across Brazilian municipalities and to test the hypothesis that forest cover would be the most influential predictor. The results indicate that the models evaluated were unable to achieve strong or consistent predictive performance, limiting their utility for identifying a single dominant driver of yield variation.

Across all modeling approaches, predictive accuracy was modest, with the best-performing models achieving test-set R^2 values of approximately 0.25–0.30. While nonlinear models such as Support Vector Regression and Random Forest Regression outperformed linear models, their performance remained limited. Residual diagnostics suggested that errors from the nonlinear models were approximately symmetric. However, formal normality assessment using QQ-plots was only conducted for the linear regression models. For these models, departures from normality suggest violations of linear model assumptions and indicate that important structure in the data remains unexplained. Linear regression models produced the lowest R^2 values, reinforcing that linear relationships alone are insufficient to capture the complexity of yield dynamics.

Interpretability analyses using SHAP further support these findings. Although several features, particularly geographic identifiers, temperature-related variables, and selected principal components, were influential in certain models, these effects were not consistent across modeling approaches or dataset variants. Forest cover did not emerge as a dominant predictor in any of the models evaluated. In the original feature space, forest cover contributed minimally to model output, and in PCA-based models, it did not substantially load onto the principal components most relevant for prediction. These results provide insufficient evidence to support the hypothesis that forest cover is the primary driver of mean coffee yield in this dataset.

Several limitations may explain these results. Coffee yield is likely influenced by complex, nonlinear interactions among climatic, management, and socioeconomic factors that are

difficult to capture using municipality-level aggregate data. In addition, strong spatial structure in the data, evidenced by the high SHAP importance of geographic identifiers such as `codigo_ibg`, may obscure underlying causal relationships, leading models to rely on location-based proxies rather than mechanistic drivers. Finally, dimensionality reduction techniques such as PCA, while useful for mitigating multicollinearity, may remove meaningful ecological signal, particularly when applied aggressively or in combination with feature filtering.

Future work should explore more flexible modeling frameworks, including neural networks and gradient boosting methods, which may better capture nonlinear relationships and interactions among environmental variables. Explicitly incorporating spatial structure through spatial or hierarchical modeling approaches may also help disentangle geographic effects from underlying ecological drivers. Alternative dimensionality reduction or feature selection methods could further be evaluated to retain predictive signal while addressing multicollinearity.

References

González-Chaves, Adrian, et al. "Positive Forest Cover Effects on Coffee Yields Are Consistent Across Regions." Zenodo (CERN European Organization for Nuclear Research), 8 Oct. 2021, <https://doi.org/10.5061/dryad.612jm644g>.

Yadav, Amit. "SHAP Values vs Feature Importance." Medium, 19 Sept. 2024, medium.com/biased-algorithms/shap-values-vs-feature-importance-ba6b91c16319.

Kusawa, Sunny, and Sunny Kusawa. "When to Perform Feature Scaling? Before Dataset Split or After? And Why?" Data Magic AI Blog, 26 July 2023, datamagiclab.com/when-to-perform-feature-scaling.

Appendix A

```
In [ ]: # import Libraries
from sklearn.preprocessing import LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import tqdm as notebook_tqdm
import shap
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor
import statsmodels.api as sm
```

```
In [ ]: # read in the data and take a look at it
# change path to fit your needs
file_path = r"C:\Users\tiffa\Machine_Learning_Final\ML\Final\cleaned_coffee.csv"

#file_path = r'/Users/adia/PyCharmMiscProject/ML/Final/cleaned_coffee.csv'

# Read the CSV
df = pd.read_csv(file_path, sep = ';') # semicolon delimited file

# Take a quick look at the data
print(df.head())
```

Exploratory Analysis

```
In [ ]: # Label Encoding
categorical_cols = df.select_dtypes(include=['object', 'category']).columns
categorical_cols
df_encoded = df.copy() # copy the dataframe

label = LabelEncoder() # Label encode each column that is non numeric
for col in categorical_cols:
    df_encoded[col] = label.fit_transform(df_encoded[col].astype(str))

df_encoded = df_encoded.drop(columns=["ID"]) # don't need the ID column
# save this
df_encoded.to_csv("df_encoded.csv", index=False)
```

```
In [ ]: # Exploratory analysis
# Correlation
corr = df_encoded.corr()
corr.head() # just to take a look
plt.figure(figsize=(14, 12)) # set figure size
sns.heatmap(corr, cmap= "coolwarm", annot= False)
plt.title("Correlation Heatmap of the Label Encoded Data")
#plt.savefig(r"C:\Users\tiffa\Machine_Learning_Final\ML\Final\Figures\correlation_h
plt.show()

# Feature histogram
df_encoded.hist(figsize= (30,30))
plt.title("Feature Distributions")
#plt.savefig(r'/Users/adia/PyCharmMiscProject/ML/Final/Figures/feature_dist_histogr
plt.show()
```

```
In [ ]: # Log transform skewed features
# function to detect skewness- log transformations cannot change left skew!
def detect_skewed_features(df, skew_threshold=1.0):
    skewed_features = []

    for col in df.columns:
        series = df[col]
```

```

    # Skip binary or near-constant
    if series.nunique() <= 3:
        continue
    # Skip negative values (Log1p cannot handle)
    if (series < 0).any():
        continue
    # Original skewness
    orig_skew = series.skew()
    # Must be skewed enough to consider transforming
    if abs(orig_skew) <= skew_threshold:
        continue
    # Skewness after Log1p
    log_skew = np.log1p(series).skew()
    # Only transform if log reduces skewness
    if abs(log_skew) < abs(orig_skew):
        skewed_features.append(col)

    return skewed_features
# Let's see the skewed columns
skewed_cols = detect_skewed_features(df_encoded)

# just in case keep the non-skewed columns separately
non_skewed_cols = [c for c in df_encoded.columns if c not in skewed_cols]

print("Skewed columns to log-transform:")
print(skewed_cols)
len(skewed_cols)

# apply log1p to skewed features
df_log = df_encoded.copy() # copy the dataset over

df_log[skewed_cols] = np.log1p(df_log[skewed_cols])

# define X and y, we will be using these from here on forward!!!!
y = df_log["yield.mean"]
X = df_log.drop(columns=["yield.mean"])

# Lets replot this histogram to see if the feature skew is better
df_log.iloc[:, :52].hist(figsize=(30,30))
plt.title("Feature Distributions After Log Transformation")
plt.savefig(r"C:\Users\tiffa\Machine_Learning_Final\ML\Final\Figures\feature_log_hi
plt.show()

```

```

In [ ]: # Exploratory Analysis
# PCA, using the encoded and log transformed dataset
# first scale, only want to look at the predictors
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Fit PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# save as a dataframe
pca_columns = [f"PC{i+1}" for i in range(X_pca.shape[1])]
df_pca = pd.DataFrame(X_pca, columns=pca_columns)

```

```

# Add the target column back
# This is for our visualization plots, it was not used to compute the PC's
df_pca["yield.mean"] = y.values

df_pca.plot(kind='scatter', x='PC1', y='PC2', c= df_pca["yield.mean"], cmap='virid
plt.title('Scatter Plot of Data Using PC1/PC2')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.grid(True)
#plt.savefig(r"C:\Users\tiffa\Machine_Learning_Final\ML\Final\Figures\pca_scatter.p
plt.show()

# 3D plot
from mpl_toolkits.mplot3d import Axes3D

# assign the definitions of each quartile
lowbound = df_pca['yield.mean'].min()
q1 = df_pca['yield.mean'].quantile(.25)
q2 = df_pca['yield.mean'].quantile(.5)
q3 = df_pca['yield.mean'].quantile(.75)
highbound = df_pca['yield.mean'].max()

# separate by quartile function
def assign_quartile(value):
    if value <= q1:
        return 0
    elif value <= q2:
        return 1
    elif value <= q3:
        return 2
    else:
        return 3

colors = df_pca['yield.mean'].apply(assign_quartile) # color code the quartiles

# plot

fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')
scatplt = ax.scatter(df_pca['PC1'], df_pca['PC2'], df_pca['PC3'], c=colors, cmap='v

cbar = plt.colorbar(scatplt, ax=ax, ticks=[0, 1, 2, 3],shrink=0.4, pad=.1)
cbar.set_label('Yield Quartile')
cbar.set_ticklabels(['Q1 (lowest)', 'Q2', 'Q3', 'Q4 (highest)'])

ax.set_xlabel('PC1')
ax.set_ylabel('PC2',labelpad=10)
ax.set_zlabel('PC3')
ax.set_title('Samples colored by Yield Mean Quartiles on the top 3 PC Axes')

#plt.savefig(r"C:\Users\tiffa\Machine_Learning_Final\ML\Final\Figures\pca_scatter3D
plt.show()

# see if any of the PC's have a correlation with the yield mean
# Compute correlation between each PC and the target

```

```

pc_target_corr = df_pca.drop("yield.mean", axis=1).corrwith(df_pca["yield.mean"])
pc_target_corr
# output was not promising

# total variance plots
plt.figure(figsize=(10,5))
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o') # plot the variance
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Variance Explained by PCA")
plt.grid(True)
#plt.savefig(r"C:\Users\tiffa\Machine_Learning_Final\ML\Final\Figures\pca_cumulativ
plt.show()

```

Data Processing

```

In [ ]: # Prepare out datasets to be used
# Split and scale
# train, test split - 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state= 32 # this is for reproducibility purposes
)
# no validation set because we will include this in the grid search

# scale the split data
scaler = StandardScaler()

# *****FIRST DATASET*****
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# PCA dataset creation
# scale first
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # scale on the unscaled dataset
X_test_scaled = scaler.transform(X_test)

# PCA transformation to the scaled dataset
pca = PCA()
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# make them into dataframes
pca_cols = [f"PC{i+1}" for i in range(X_train_pca.shape[1])]

X_train_pca = pd.DataFrame(X_train_pca, columns=pca_cols, index=X_train.index)
X_test_pca = pd.DataFrame(X_test_pca, columns=pca_cols, index=X_test.index)

# *****SECOND DATASET*****
# running on the first 7 PC's- since they explain about 75% of the variance
X_train_pca_selected = X_train_pca.iloc[:, :7]
X_test_pca_selected = X_test_pca.iloc[:, :7]

```

```

# Third dataset creation
# remove 0 variance features and highly correlated features
from sklearn.feature_selection import VarianceThreshold

# set the threshold to 0.01, if its less than that then we remove
vt = VarianceThreshold(threshold=0.01)
X_vt = vt.fit_transform(X)

# get list of retained column names
cols_vt = X.columns[vt.get_support()]
X_vt_df = pd.DataFrame(X_vt, columns=cols_vt)

# remove the highly correlated features
corr = X_vt_df.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))

to_drop = [col for col in upper.columns if any(upper[col] > 0.90)]
print(to_drop)

X_reduced = X_vt_df.drop(columns=to_drop)

# split scale run PCA on this reduced features
X_train_red, X_test_red, y_train_red, y_test_red = train_test_split(
    X_reduced, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_red_scaled = scaler.fit_transform(X_train_red)
X_test_Red_scaled = scaler.transform(X_test_red)

pca = PCA()
X_train_pca_red = pca.fit_transform(X_train_red_scaled)
X_test_pca_red = pca.transform(X_test_Red_scaled)

# *****THIRD DATASET*****
# run on the first 7 PC's like before
X_train_red_selected = X_train_pca_red[:, :7]
X_test_red_selected = X_test_pca_red[:, :7]

# Convert PCA arrays to DataFrames so SHAP can use .sample()
pc_names = [f"PC{i+1}" for i in range(7)]

X_train_red_selected = pd.DataFrame(X_train_red_selected, columns=pc_names)
X_test_red_selected = pd.DataFrame(X_test_red_selected, columns=pc_names)

```

```

In [ ]: # see what features contribute to PC2
# Get PCA Loadings (components)
loadings = pd.DataFrame(
    pca.components_,
    columns=X_train.columns,
    index=[f"PC{i+1}" for i in range(pca.n_components_)]
)

# Sort PC2 features by absolute contribution, but keep actual signed loadings
pc2_sorted = loadings.loc["PC2"].reindex(

```



```

        loadings.loc["PC2"].abs().sort_values(ascending=False).index
    )

    print(pc2_sorted.head(10))  # top 10 features
    # PCA Loadings
    loadings = pd.DataFrame(
        pca.components_,
        columns=X_train.columns,
        index=[f"PC{i+1}" for i in range(pca.n_components_)]
    )

    # Sort by absolute loading values (strongest contributors)
    pc2_sorted = loadings.loc["PC2"].sort_values(key=lambda x: np.abs(x), ascending=False)

    # Take top 10 contributors
    top_pc2 = pc2_sorted.head(10)

    # Plot
    plt.figure(figsize=(8, 6))
    top_pc2[::-1].plot(kind="barh")  # reverse for descending order visually
    plt.title("Top Feature Contributions to PC2")
    plt.xlabel("Loading Weight")
    plt.ylabel("Feature")
    plt.tight_layout()
    plt.show()

    # See what features contribute to PC2 for the Reduced dataset
    # get Loadings
    loadings_red = pd.DataFrame(
        pca.components_,
        columns=X_train_red.columns,  # original reduced feature names
        index=[f"PC{i+1}" for i in range(pca.n_components_)]
    )

    # print
    pc2_loadings = loadings_red.loc["PC2"].sort_values(key=lambda x: abs(x), ascending=False)
    print("PC2 Loadings (Reduced-Feature PCA):")
    print(pc2_loadings.head(15))  # top 15 contributors

    # plot
    plt.figure(figsize=(8, 6))
    pc2_loadings.head(15).plot(kind='barh')
    plt.xlabel("Loading Weight")
    plt.title("Top PC2 Loadings (Reduced-Feature PCA)")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.show()

```

Random Forest Regression

```

In [ ]:  # Defining the fixed parameters of RFR model
rfr_model = RandomForestRegressor(random_state=8, oob_score=True)

        # Parameter grid to be tested in GridSearchCV
rfr_param_grid = { 'criterion':['squared_error', 'absolute_error', 'friedman_mse'],

```

```

        'max_features' : [None, 'sqrt', 'log2'] }

parameter_tests = GridSearchCV(
    estimator=rfr_model,
    param_grid=rfr_param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1,
    verbose=2)

```

```

In [ ]: # *** FIRST DATASET VARIANT ***
parameter_tests.fit(X_train_scaled, y_train)

    # best model scoring metrics
print("Best R²:", parameter_tests.best_score_)
print("Best Params:", parameter_tests.best_params_)

    # applying best model to test set
optimized_rfr_model = parameter_tests.best_estimator_
rfr_y_pred = optimized_rfr_model.predict(X_test_scaled)

    # model performance
print("Test R²:", r2_score(y_test, rfr_y_pred))
print("Test MAE:", mean_absolute_error(y_test, rfr_y_pred))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, rfr_y_pred)))

    # creating SHAP summary
rfr_explainer = shap.TreeExplainer(optimized_rfr_model)
rfr_shap_values = rfr_explainer.shap_values(X_test_scaled)
shap.summary_plot(rfr_shap_values, X_test_scaled, feature_names=X.columns, show = F
plt.tight_layout()
#plt.savefig(r"/Users/adia/PyCharmMiscProject/ML/Final/Figures/RFR_shap_sum.png", bb

```

```

In [ ]: # *** SECOND DATASET VARIANT***
parameter_tests.fit(X_train_pca_selected, y_train)

    # best model scoring metrics
print("Best R²:", parameter_tests.best_score_)
print("Best Params:", parameter_tests.best_params_)

    # applying best model to test set
pca_optimized_rfr_model = parameter_tests.best_estimator_
pca_rfr_y_pred = pca_optimized_rfr_model.predict(X_test_pca_selected)

    # model performance
print("Test R²:", r2_score(y_test, pca_rfr_y_pred))
print("Test MAE:", mean_absolute_error(y_test, pca_rfr_y_pred))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, pca_rfr_y_pred)))

    # generating SHAP summary
rfr_explainer = shap.TreeExplainer(pca_optimized_rfr_model)
rfr_shap_values = rfr_explainer.shap_values(X_test_pca_selected)
shap.summary_plot(rfr_shap_values, X_test_pca_selected, feature_names=X_train_pca_s
plt.tight_layout()
#plt.savefig(r"/Users/adia/PyCharmMiscProject/ML/Final/Figures/RFR_LPS_shap_sum.png

```

```
In [ ]: # *** THIRD DATASET VARIANT***
parameter_tests.fit(X_train_red_selected, y_train)

# best model scoring metrics
print("Best R²:", parameter_tests.best_score_)
print("Best Params:", parameter_tests.best_params_)

# applying best model to test set
red_pca_optimized_rfr_model = parameter_tests.best_estimator_
pca_red_rfr_y_pred = red_pca_optimized_rfr_model.predict(X_test_red_selected)

# model performance
print("Test R²:", r2_score(y_test, pca_red_rfr_y_pred))
print("Test MAE:", mean_absolute_error(y_test, pca_red_rfr_y_pred))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, pca_red_rfr_y_pred)))

# generating SHAP summary
rfr_explainer = shap.TreeExplainer(red_pca_optimized_rfr_model)
rfr_shap_values = rfr_explainer.shap_values(X_test_red_selected)
shap.summary_plot(rfr_shap_values, X_test_red_selected, feature_names=X_train_red_s
plt.tight_layout()
#plt.savefig(r"/Users/adia/PyCharmMiscProject/ML/Final/Figures/RFR_LPS_red_shap_sum
```

```
In [ ]: # generating residuals plots
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.scatterplot(x=rfr_y_pred, y=y_test, color = "olive", ax = axes[0])
axes[0].set_xlabel("Predicted Mean Yield")
axes[0].set_ylabel("Residuals")
axes[0].set_title('Log Trans. and Scaled RFR Residuals')
axes[0].grid(True)

sns.residplot(x=pca_rfr_y_pred, y=y_test, color = 'magenta', ax = axes[1])
axes[1].set_xlabel("Predicted Mean Yield")
axes[1].set_ylabel("Residuals")
axes[1].set_title(' Scaled, Log + PCA Trans RFR Residuals')
axes[1].grid(True)

sns.residplot(x=pca_red_rfr_y_pred, y=y_test, color = 'blue', ax = axes[2])
axes[2].set_xlabel("Predicted Mean Yield")
axes[2].set_ylabel("Residuals")
axes[2].set_title(' Scaled, Log + PCA Reduced RFR Residuals')
axes[2].grid(True)

#plt.savefig(r"/Users/adia/PyCharmMiscProject/ML/Final/Figures/RFR_resid_multi.png")
plt.show()
```

```
In [ ]: # generating QQ-plots
fig, axes = plt.subplots(1, 3, figsize=(20, 5))

sm.qqplot(data=(y_test - rfr_y_pred), line='45', ax = axes[0])
axes[0].set_title('Log Trans. and Scaled Residuals QQ Plot')

sm.qqplot(data=(y_test - pca_rfr_y_pred), line='45', ax = axes[1])
axes[1].set_title('Scaled, Log + PCA Trans Residuals QQ Plot')
```

```
sm.qqplot(data=(y_test - pca_red_rfr_y_pred), line='45', ax = axes[2])
axes[2].set_title('Scaled, Log + PCA Reduced Residuals QQ Plot')

#plt.savefig(r"/Users/adia/PyCharmMiscProject/ML/Final/Figures/RFR_qq_multi.png")
plt.show()
```

Support Vector Regression

```
In [ ]: # Support Vector Regression Code
# scale the data and fit the model
# ***** First Dataset*****

svr_model = Pipeline([ # use pipeline to scale and train the data
    ("scaler", StandardScaler()), # scale after splitting to prevent data leakage!
    ("svr", SVR(kernel="rbf")) # chose RBF kernel since the relationship between X
])
#hyperparameter grid search for the best parameters
param_grid = {
    "svr__C": [0.1, 1, 10, 100],
    "svr__gamma": ["scale", "auto", 0.01, 0.001, 0.0001],
    "svr__epsilon": [0.1, 0.5, 1.0]
}

grid = GridSearchCV(
    estimator=svr_model,
    param_grid=param_grid,
    cv=5, # cross validation
    scoring="r2",
    n_jobs=-1,
    verbose=2
)

# fit this grid only on the training data
grid.fit(X_train, y_train) # since we are scaling in the pipeline we won't use the

print("Best R2 training:", grid.best_score_)
print("Best Params:", grid.best_params_)

best_svr_model = grid.best_estimator_ # assign out best

# evaluate on the test set
y_pred = grid.predict(X_test)

print("Test R2:", r2_score(y_test, y_pred))
print("Test MAE:", mean_absolute_error(y_test, y_pred))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

# SHAP figure code
explainer = shap.KernelExplainer(grid.predict, X_train.sample(50))
shap_values = explainer.shap_values(X_test.sample(50))

shap.summary_plot(shap_values, X_test.sample(50))

#residuals Code
import statsmodels as sm
residuals = y_test - y_pred
```

```

# residuals
sns.residplot(x=y_pred, y=residuals, lowess=True)
plt.xlabel("Predicted Mean Yield")
plt.ylabel("Residuals")
plt.title("SVR Residual Plot (Log-Transformed & Scaled Data)")
plt.grid(True)
plt.tight_layout()
plt.savefig("svr_residual_plot.png", dpi=300)
plt.show()

# qq
sm.qqplot(residuals, line='45')
plt.title("SVR Residuals QQ Plot (Log-Transformed & Scaled Data)")
plt.tight_layout()
plt.savefig("svr_qq_plot.png", dpi=300)
plt.show()

# *****Second Dataset*****
# fit the model
svr_model_pca = SVR(kernel="rbf")

#hyper parameters
param_grid_pca = {
    "C": [1, 10, 100, 1000, 10000],
    "gamma": [1, 0.5, 0.1, 0.01, 0.001],
    "epsilon": [0.001, 0.01, 0.1]
}

# grid search
grid = GridSearchCV(
    estimator=svr_model_pca,
    param_grid= param_grid_pca,
    cv= 5,
    scoring = "r2",
    n_jobs=1,
)

# fit this grid only on the training data
grid.fit(X_train_pca_selected, y_train)
# PCA is only for the predictors

print("Best R²:", grid.best_score_)
print("Best Params:", grid.best_params_)

best_svr_pca = grid.best_estimator_

# evaluate on the test set
y_pred_pca = grid.predict(X_test_pca_selected)

print("Test R²:", r2_score(y_test, y_pred_pca))
print("Test MAE:", mean_absolute_error(y_test, y_pred_pca))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_pca)))

# shap values on the PCA set:
explainer = shap.KernelExplainer(grid.predict, X_train_pca_selected.sample(50))
shap_values = explainer.shap_values(X_test_pca_selected.sample(50))

```

```

shap.summary_plot(shap_values, X_test_pca_selected.sample(50))
# residuals for PCA model
residuals_pca = y_test - y_pred_pca

# residual
sns.residplot(x=y_pred_pca, y=residuals_pca, lowess=True)
plt.xlabel("Predicted Mean Yield (PCA SVR)")
plt.ylabel("Residuals")
plt.title("SVR Residual Plot (PCA Transformed Data)")
plt.grid(True)
plt.tight_layout()
plt.savefig("svr_pca_residual_plot.png", dpi=300)
plt.show()

# qq
sm.qqplot(residuals_pca, line='45')
plt.title("SVR Residuals QQ Plot (PCA Transformed Data)")
plt.tight_layout()
plt.savefig("svr_pca_residual_qq_plot.png", dpi=300)
plt.show()

# *****Third Dataset*****
# fit the model
svr_model_pca_red = SVR(kernel="rbf")

#hyper parameters
param_grid_pca_red = {
    "C": [10, 100, 1000, 10000],
    "gamma": [ 1, 0.1, 0.01, 0.001],
    "epsilon": [0.01, 0.05, 0.1]
}

# grid search
grid = GridSearchCV(
    estimator=svr_model_pca_red,
    param_grid= param_grid_pca_red,
    cv= 5,
    scoring = "r2",
    n_jobs=1,
)
# fit this grid only on the training data
grid.fit(X_train_red_selected, y_train)
# PCA is only for the predictors

print("Best R²:", grid.best_score_)
print("Best Params:", grid.best_params_)

best_svr_pca = grid.best_estimator_
# evaluate on the test set
y_pred_red = grid.predict(X_test_red_selected)

print("Test R²:", r2_score(y_test, y_pred_red))
print("Test MAE:", mean_absolute_error(y_test, y_pred_red))
print("Test RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_red)))

```

```
#Residuals
# Calculate residuals
residuals_red = y_test - y_pred_red

# residual
sns.residplot(x=y_pred_red, y=residuals_red, lowess=True)
plt.xlabel("Predicted Mean Yield (PCA Reduced SVR)")
plt.ylabel("Residuals")
plt.title("SVR Residual Plot (Reduced and PCA Transformed Data)")
plt.grid(True)
plt.tight_layout()
plt.savefig("svr_pca_residual_plot.png", dpi=300)
plt.show()

# shap on the features
explainer = shap.KernelExplainer(grid.predict, X_train_red_selected.sample(50))
shap_values = explainer.shap_values(X_test_red_selected.sample(50))

shap.summary_plot(shap_values, X_test_red_selected.sample(50))
```